

```

"""
Completed controller.

Sequence:
1. Open gripper and raise arm.
2. Navigate to the stick base.
3. Grab the stick.
4. Navigate to the puck.
5. Wind up, lower the stick, and shoot.
"""

import math
import time

import rclpy
from geometry_msgs.msg import PoseStamped, Twist, Vector3
from rclpy.action import ActionClient
from rclpy.node import Node
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy
from robomaster_msgs.action import GripperControl

# =====
# STAGE 1: NAVIGATE TO THE BASE AND GRAB
# =====

# -----
# Parameters
# -----

# Robot ID
LAB_ROBOT_ID = 9

# Stick ID
GRAB_STICK_ID = 4

# Final distance from the base edge
GRAB_FINAL_EDGE_CLEARANCE = 0.43

# Lateral path correction
# +: offset shift right
# -: offset shift left
GRAB_LATERAL_OFFSET = -0.10

GRAB_GRIPPER_OPEN = 1
GRAB_GRIPPER_CLOSE = 2
GRAB_GRIPPER_POWER = 0.5

GRAB_ARM_SPEED = 0.08
GRAB_ARM_PUBLISH_RATE = 20.0

GRAB_RAISE_Z_SPEED = GRAB_ARM_SPEED
GRAB_LOWER_Z_SPEED = -GRAB_ARM_SPEED

GRAB_PREPARE_RAISE_TIME = 1.0

# Grab motion
GRAB_GRAB_FORWARD_SPEED = 0.04
GRAB_GRAB_DOWN_SPEED = -GRAB_ARM_SPEED
GRAB_GRAB_DIAGONAL_TIME = 1.0

# Return motion after grabbing
GRAB_GRAB_RETURN_X_SPEED = -GRAB_GRAB_FORWARD_SPEED
GRAB_GRAB_RETURN_Z_SPEED = GRAB_ARM_SPEED
GRAB_GRAB_RETURN_TIME = GRAB_GRAB_DIAGONAL_TIME

class NavigateAndGrabStick(Node):

    def __init__(self):
        super().__init__(
            f"navigate_and_grab_stick_robot{LAB_ROBOT_ID}_stick{GRAB_STICK_ID}"
        )

        if GRAB_STICK_ID not in (1, 2, 3, 4):
            raise ValueError("STICK_ID must be 1, 2, 3, 4")

        self.robot_id = LAB_ROBOT_ID
        self.stick_id = GRAB_STICK_ID

        self.robot_pose = None
        self.base_pose = None

        # -----
        # Stick-base
        # -----

        self.stick_pose_topic = (
            f"/vrpn_mocap/hockey_sticks_{self.stick_id}/pose"
        )

        # Which local base axis points toward the three-marker edge.
        self.THREE_MARKER_EDGE_AXIS = "x"
        self.THREE_MARKER_EDGE_SIGN = 1.0

        # Base half-length
        self.BASE_HALF_LENGTH = 0.15

        # Pre-approach distance
        self.PRE_APPROACH_CLEARANCE = 1.00

        # Final clearance from the selected edge.
        # This parameter moves the final stopping line forward/backward.
        self.GRAB_FINAL_EDGE_CLEARANCE = float(GRAB_FINAL_EDGE_CLEARANCE)

        # Lateral target correction in metres.
        # +: shifts the target to the right.
        # -: shifts the target to the left.
        self.FINAL_LATERAL_OFFSET = float(GRAB_LATERAL_OFFSET)

        # Apply the same lateral correction to the pre-approach point so the
        # final straight segment stays parallel to the base normal.
        self.PRE_APPROACH_LATERAL_OFFSET = float(GRAB_LATERAL_OFFSET)

        # -----
        # Navigation parameters
        # -----

        self.lookahead_l = 0.10
        self.kp_position = 0.55
        self.max_linear_speed = 0.30
        self.max_angular_speed = 0.75

```

```

self.pre_approach_tolerance = 0.10
self.heading_tolerance = math.radians(4.0)

self.kp_heading = 1.5
self.max_align_angular_speed = 0.60

# Final straight approach
self.max_final_approach_speed = 0.12
self.min_final_approach_speed = 0.025
self.final_speed_kp = 0.60

self.final_heading_kp = 0.8
self.max_final_heading_correction = 0.20

self.stop_line_margin = 0.025

# Tolerance.
self.final_position_tolerance = 0.05
self.max_safe_lateral_error = 0.4

self.stage = "waiting_for_preparation"

self.pre_target = None
self.final_target = None
self.approach_heading = None
self.last_status = None

best_effort_qos = QoSProfile(
    reliability=ReliabilityPolicy.BEST_EFFORT,
    durability=DurabilityPolicy.VOLATILE,
    depth=1,
)

self.cmd_pub = self.create_publisher(
    Twist,
    f"/robot{self.robot_id}/cmd_vel",
    10,
)

self.arm_pub = self.create_publisher(
    Vector3,
    f"/robot{self.robot_id}/cmd_arm",
    10,
)

self.gripper_client = ActionClient(
    self,
    GripperControl,
    f"/robot{self.robot_id}/gripper",
)

self.create_subscription(
    PoseStamped,
    f"/vrpn_mocap/dji_robot_{self.robot_id}/pose",
    self.robot_pose_callback,
    best_effort_qos,
)

self.create_subscription(
    PoseStamped,
    self.stick_pose_topic,
    self.base_pose_callback,
    best_effort_qos,
)

self.timer = self.create_timer(0.05, self.control_loop)

self.get_logger().info(
    f"Controller created for robot {self.robot_id}, stick {self.stick_id}."
)
self.get_logger().info(
    f"Stick topic: {self.stick_pose_topic}"
)
self.get_logger().info(
    f"Final edge clearance: {self.GRAB_FINAL_EDGE_CLEARANCE:.3f} m"
)
self.get_logger().info(
    f"Lateral offset: {self.FINAL_LATERAL_OFFSET:.3f} m"
)

# -----
# Poses
# -----

def robot_pose_callback(self, msg):
    self.robot_pose = msg

def base_pose_callback(self, msg):
    self.base_pose = msg

@staticmethod
def yaw_from_quaternion(q):
    siny_cosp = 2.0 * (q.w * q.z + q.x * q.y)
    cosy_cosp = 1.0 - 2.0 * (q.y * q.y + q.z * q.z)
    return math.atan2(siny_cosp, cosy_cosp)

@staticmethod
def wrap_angle(angle):
    return math.atan2(math.sin(angle), math.cos(angle))

@staticmethod
def clamp(value, minimum, maximum):
    return max(min(value, maximum), minimum)

def robot_xytheta(self):
    x = self.robot_pose.pose.position.x
    y = self.robot_pose.pose.position.y
    theta = self.yaw_from_quaternion(
        self.robot_pose.pose.orientation
    )
    return x, y, theta

def log_once(self, message):
    if message != self.last_status:
        self.get_logger().info(message)
        self.last_status = message

# -----
# Arm & gripper action
# -----

def stop_robot(self):

```

```

        self.cmd_pub.publish(Twist())

def hard_stop_robot(self, repetitions=8, delay=0.02):
    """
    Publish several zero-velocity commands to make the stop command robust.
    """
    for _ in range(repetitions):
        self.stop_robot()
        time.sleep(delay)

def stop_arm(self):
    self.arm_pub.publish(Vector3())

def move_arm_xyz(self, x_speed, y_speed, z_speed, duration):
    cmd = Vector3()
    cmd.x = float(x_speed)
    cmd.y = float(y_speed)
    cmd.z = float(z_speed)

    period = 1.0 / GRAB_ARM_PUBLISH_RATE
    end_time = time.monotonic() + duration

    try:
        while rclpy.ok() and time.monotonic() < end_time:
            self.arm_pub.publish(cmd)
            time.sleep(period)
    finally:
        self.stop_arm()
        time.sleep(0.3)

def move_arm_z(self, z_speed, duration):
    self.move_arm_xyz(
        x_speed=0.0,
        y_speed=0.0,
        z_speed=z_speed,
        duration=duration,
    )

def set_gripper(self, target_state):
    if not self.gripper_client.wait_for_server(timeout_sec=10.0):
        self.get_logger().error(
            f"Gripper action server unavailable: "
            f"/robot{self.robot_id}/gripper"
        )
        return False

    goal = GripperControl.Goal()
    goal.target_state = int(target_state)
    goal.power = float(GRAB_GRIPPER_POWER)

    send_future = self.gripper_client.send_goal_async(goal)
    rclpy.spin_until_future_complete(
        self,
        send_future,
        timeout_sec=5.0,
    )

    if not send_future.done() or send_future.result() is None:
        self.get_logger().error(
            "No response from gripper action server."
        )
        return False

    goal_handle = send_future.result()

    if not goal_handle.accepted:
        self.get_logger().error(
            "Gripper command was rejected."
        )
        return False

    result_future = goal_handle.get_result_async()
    rclpy.spin_until_future_complete(
        self,
        result_future,
        timeout_sec=5.0,
    )

    return True

# -----
# Grabbing sequence
# -----

def prepare_arm_for_navigation(self):
    self.hard_stop_robot()
    self.stop_arm()

    self.get_logger().info(
        "Preparation step 1: opening gripper."
    )

    if not self.set_gripper(GRAB_GRIPPER_OPEN):
        return False

    time.sleep(0.5)

    self.get_logger().info(
        "Preparation step 2: raising arm."
    )

    self.move_arm_z(
        GRAB_RAISE_Z_SPEED,
        GRAB_PREPARE_RAISE_TIME,
    )

    self.stop_arm()

    self.get_logger().info(
        "Arm raised and gripper left open."
    )
    return True

def grab_stick(self):
    self.hard_stop_robot()
    self.stop_arm()

    self.get_logger().info(
        "Grab step 1: confirming gripper is open."
    )

    if not self.set_gripper(GRAB_GRIPPER_OPEN):
        return False

```

```

time.sleep(0.3)

self.get_logger().info(
    "Grab step 2: moving arm diagonally forward and downward."
)

self.move_arm_xyz(
    x_speed=GRAB_GRAB_FORWARD_SPEED,
    y_speed=0.0,
    z_speed=GRAB_GRAB_DOWN_SPEED,
    duration=GRAB_GRAB_DIAGONAL_TIME,
)

self.get_logger().info(
    "Grab step 3: closing gripper."
)

if not self.set_gripper(GRAB_GRIPPER_CLOSE):
    return False

time.sleep(0.8)

self.get_logger().info(
    "Grab step 4: moving arm diagonally backward and upward "
    "with the stick."
)

self.move_arm_xyz(
    x_speed=GRAB_GRAB_RETURN_X_SPEED,
    y_speed=0.0,
    z_speed=GRAB_GRAB_RETURN_Z_SPEED,
    duration=GRAB_GRAB_RETURN_TIME,
)

self.stop_arm()
self.get_logger().info(
    "Navigation and diagonal grab sequence completed."
)
return True

# -----
# Target calculation
# -----

def get_three_marker_edge_direction(self):
    base_yaw = self.yaw_from_quaternion(
        self.base_pose.pose.orientation
    )

    if self.THREE_MARKER_EDGE_AXIS == "x":
        dx = math.cos(base_yaw)
        dy = math.sin(base_yaw)
    elif self.THREE_MARKER_EDGE_AXIS == "y":
        dx = -math.sin(base_yaw)
        dy = math.cos(base_yaw)
    else:
        raise ValueError(
            'THREE_MARKER_EDGE_AXIS must be "x" or "y".'
        )

    dx *= self.THREE_MARKER_EDGE_SIGN
    dy *= self.THREE_MARKER_EDGE_SIGN
    return dx, dy

def calculate_targets(self):
    base_x = self.base_pose.pose.position.x
    base_y = self.base_pose.pose.position.y

    # Outward normal
    dx, dy = self.get_three_marker_edge_direction()
    self.approach_heading = math.atan2(-dy, -dx)

    right_x = math.sin(self.approach_heading)
    right_y = -math.cos(self.approach_heading)

    pre_distance = (
        self.BASE_HALF_LENGTH
        + self.PRE_APPROACH_CLEARANCE
    )

    final_distance = (
        self.BASE_HALF_LENGTH
        + self.GRAB_FINAL_EDGE_CLEARANCE
    )

    self.pre_target = (
        base_x
        + pre_distance * dx
        + self.PRE_APPROACH_LATERAL_OFFSET * right_x,
        base_y
        + pre_distance * dy
        + self.PRE_APPROACH_LATERAL_OFFSET * right_y,
    )

    self.final_target = (
        base_x
        + final_distance * dx
        + self.FINAL_LATERAL_OFFSET * right_x,
        base_y
        + final_distance * dy
        + self.FINAL_LATERAL_OFFSET * right_y,
    )

    self.get_logger().info(
        "Targets calculated: "
        f"pre=({self.pre_target[0]:.3f}, "
        f"{self.pre_target[1]:.3f}), "
        f"final=({self.final_target[0]:.3f}, "
        f"{self.final_target[1]:.3f}), "
        f"heading={math.degrees(self.approach_heading):.1f} deg"
    )

# -----
# Navigation
# -----

def drive_to_point(self, target_x, target_y, tolerance):
    x, y, theta = self.robot_xytheta()

    px = x + self.lookahead_l * math.cos(theta)
    py = y + self.lookahead_l * math.sin(theta)

```

```

ex = target_x - px
ey = target_y - py
distance = math.hypot(ex, ey)

if distance < tolerance:
    self.stop_robot()
    return True

pdot_x = self.kp_position * ex
pdot_y = self.kp_position * ey

v = (
    math.cos(theta) * pdot_x
    + math.sin(theta) * pdot_y
)

w = (
    -math.sin(theta) * pdot_x
    + math.cos(theta) * pdot_y
) / self.lookahead_l

v = self.clamp(
    v,
    -self.max_linear_speed,
    self.max_linear_speed,
)

w = self.clamp(
    w,
    -self.max_angular_speed,
    self.max_angular_speed,
)

cmd = Twist()
cmd.linear.x = v
cmd.angular.z = w
self.cmd_pub.publish(cmd)

return False

def align_to_heading(self, desired_heading):
    _, _, theta = self.robot_xytheta()

    heading_error = self.wrap_angle(
        desired_heading - theta
    )

    if abs(heading_error) < self.heading_tolerance:
        self.stop_robot()
        return True

    w = self.kp_heading * heading_error
    w = self.clamp(
        w,
        -self.max_align_angular_speed,
        self.max_align_angular_speed,
    )

    cmd = Twist()
    cmd.angular.z = w
    self.cmd_pub.publish(cmd)

    return False

def drive_straight_to_final_target(self):
    x, y, theta = self.robot_xytheta()

    base_x = self.base_pose.pose.position.x
    base_y = self.base_pose.pose.position.y

    dx, dy = self.get_three_marker_edge_direction()
    approach_heading = math.atan2(-dy, -dx)
    right_x = math.sin(approach_heading)
    right_y = -math.cos(approach_heading)

    target_x = base_x + self.FINAL_LATERAL_OFFSET * right_x
    target_y = base_y + self.FINAL_LATERAL_OFFSET * right_y

    dx_to_base = target_x - x
    dy_to_base = target_y - y
    distance_to_base = math.hypot(dx_to_base, dy_to_base)

    stop_distance = (
        self.BASE_HALF_LENGTH
        + self.GRAB_FINAL_EDGE_CLEARANCE
    )

    remaining_distance = distance_to_base - stop_distance

    if remaining_distance <= self.stop_line_margin:
        self.hard_stop_robot()
        self.get_logger().info(
            "Stick-centre clearance reached. "
            f"distance_to_base={distance_to_base:.3f} m, "
            f"stop_distance={stop_distance:.3f} m"
        )
        return True

    # Virtual point in front of robot
    px = x + self.lookahead_l * math.cos(theta)
    py = y + self.lookahead_l * math.sin(theta)

    ex = target_x - px
    ey = target_y - py

    pdot_x = self.kp_position * ex
    pdot_y = self.kp_position * ey

    w = (
        -math.sin(theta) * pdot_x
        + math.cos(theta) * pdot_y
    ) / self.lookahead_l

    w = self.clamp(
        w,
        -self.max_final_heading_correction,
        self.max_final_heading_correction,
    )

    v = self.final_speed_kp * remaining_distance
    v = self.clamp(
        v,

```

```

        self.min_final_approach_speed,
        self.max_final_approach_speed,
    )

    cmd = Twist()
    cmd.linear.x = v
    cmd.angular.z = w
    self.cmd_pub.publish(cmd)

    return False

def drive_backward_to_pre_approach(self):
    """
    After grabbing the stick, reverse along the same straight approach line
    until the robot returns to the original pre-approach target.
    """
    x, y, theta = self.robot_xytheta()
    target_x, target_y = self.pre_target

    error_x = target_x - x
    error_y = target_y - y
    distance = math.hypot(error_x, error_y)

    if distance <= self.pre_approach_tolerance:
        self.hard_stop_robot()
        self.get_logger().info(
            "Returned to the original pre-approach point."
        )
        return True

    heading_error = self.wrap_angle(
        self.approach_heading - theta
    )

    w = self.final_heading_kp * heading_error
    w = self.clamp(
        w,
        -self.max_final_heading_correction,
        self.max_final_heading_correction,
    )

    reverse_x = -math.cos(self.approach_heading)
    reverse_y = -math.sin(self.approach_heading)
    remaining_reverse = (
        error_x * reverse_x
        + error_y * reverse_y
    )

    if remaining_reverse <= 0.0:
        self.hard_stop_robot()
        self.get_logger().info(
            "Pre-approach stopping line reached while reversing."
        )
        return True

    speed = self.final_speed_kp * remaining_reverse
    speed = self.clamp(
        speed,
        self.min_final_approach_speed,
        self.max_final_approach_speed,
    )

    cmd = Twist()
    cmd.linear.x = -speed
    cmd.angular.z = w
    self.cmd_pub.publish(cmd)

    return False

# -----
# State machine
# -----

def control_loop(self):
    if self.stage == "waiting_for_preparation":
        self.stop_robot()
        return

    if self.stage in ("navigation_done", "done", "failed"):
        self.stop_robot()
        return

    if self.stage == "return_to_pre_approach":
        self.log_once("Stage: RETURN_TO_PRE_APPROACH")

        reached = self.drive_backward_to_pre_approach()

        if reached:
            self.stage = "returned_to_pre_approach"
            self.last_status = None
            return

    if self.stage == "returned_to_pre_approach":
        self.stop_robot()
        return

    if self.robot_pose is None:
        self.log_once("Waiting for robot pose...")
        return

    if self.base_pose is None:
        self.log_once(
            f"Waiting for stick {self.stick_id} pose..."
        )
        return

    if self.pre_target is None:
        self.calculate_targets()

    if self.stage == "go_to_pre_approach":
        self.log_once("Stage: GO_TO_PRE_APPROACH")

        reached = self.drive_to_point(
            self.pre_target[0],
            self.pre_target[1],
            self.pre_approach_tolerance,
        )

        if reached:
            self.get_logger().info(
                "Reached the pre-approach point."
            )
            self.stage = "align_to_base"

```

```

        self.last_status = None

    elif self.stage == "align_to_base":
        self.log_once("Stage: ALIGN_TO_BASE")

        x, y, _ = self.robot_xytheta()
        base_x = self.base_pose.pose.position.x
        base_y = self.base_pose.pose.position.y

        dx, dy = self.get_three_marker_edge_direction()
        approach_heading = math.atan2(-dy, -dx)

        right_x = math.sin(approach_heading)
        right_y = -math.cos(approach_heading)

        target_x = base_x + self.FINAL_LATERAL_OFFSET * right_x
        target_y = base_y + self.FINAL_LATERAL_OFFSET * right_y

        heading_to_base_centre = math.atan2(
            target_y - y,
            target_x - x,
        )

        aligned = self.align_to_heading(
            heading_to_base_centre
        )

        if aligned:
            self.get_logger().info(
                "Robot aligned directly toward the stick-base centre."
            )
            self.stage = "straight_to_base"
            self.last_status = None

    elif self.stage == "straight_to_base":
        self.log_once("Stage: STRAIGHT_TO_BASE")

        reached = self.drive_straight_to_final_target()

        if self.stage == "failed":
            return

        if reached:
            self.hard_stop_robot()
            self.get_logger().info(
                "Reached the final grab position."
            )
            self.stage = "navigation_done"
            self.last_status = None

# =====
# STAGE 2: NAVIGATE TO THE PUCK AND SHOOT
# =====

# -----
# Robot and Vicon topic configuration
# -----

# Puck color
SHOT_PUCK_COLOR = "blue"

# Goal ID
GOAL_ID = 2

# Final distance to the puck
SHOT_ROBOT_TO_PUCK_DISTANCE = 0.25

# Pre-swing point clearance
SHOT_PRE_SWING_DISTANCE = 1.00

# Select which side of the puck for shoot:
# +1 and -1 are the two opposite points on the 90-degree line.
SHOT_SHOT_SIDE = 1.0

# Stop this many metres before the exact final point, along the same
# perpendicular line. Positive means stop farther away from the puck.
SHOT_FINAL_STOP_EARLY_OFFSET = 0.05

# Shared offset along the puck-to-goal direction.
# +: shifts the right
# -: shifts the left
SHOT_SHOT_DIRECTION_OFFSET = -0.10

SHOT_ROBOT_POSE_TOPIC = f"/vrpn_mocap/dji_robot_{LAB_ROBOT_ID}/pose"
SHOT_PUCK_POSE_TOPIC = f"/vrpn_mocap/hockey_puck_{SHOT_PUCK_COLOR}/pose"
SHOT_GOAL_POSE_TOPIC = f"/vrpn_mocap/hockey_goal_{GOAL_ID}/pose"

# -----
# Shooting action
# -----

SHOT_ARM_SPEED = 0.08
SHOT_ARM_PUBLISH_RATE = 20.0
SHOT_LOWER_ARM_Z_SPEED = -SHOT_ARM_SPEED
SHOT_LOWER_STICK_TIME = 1.8

# Wind up rotation
SHOT_PRE_ROTATION_ANGLE = math.pi
SHOT_PRE_ROTATION_SPEED = 0.55

# Shooting rotation
SHOT_SHOT_ROTATION_ANGLE = 3 * math.pi / 2
SHOT_SHOT_ROTATION_SPEED = 10.0

# +1: CCW
# -1: CW
SHOT_PRE_ROTATION_SIGN = 1.0
SHOT_SHOT_ROTATION_SIGN = -1.0

SHOT_STOP_SETTLE_TIME = 0.50
SHOT_STICK_GROUND_SETTLE_TIME = 0.40

SHOT_MAX_PRE_ROTATION_TIME = 8.0
SHOT_MAX_SHOT_ROTATION_TIME = 10.0

class RotateShotController(Node):
    """
    Assumptions:
    1. The robot has already grabbed and raised the hockey stick.

```

2. The stick points approximately along the robot's forward x-axis.
3. The ceiling-camera/Vicon system publishes robot, puck, and goal poses.
4. The robot shoots by rotating in place so the stick tip sweeps through the puck with tangential velocity directed toward the goal.

"""

```
def __init__(self):
    super().__init__(f"rotate_shot_controller_robot{LAB_ROBOT_ID}")

    self.robot_id = LAB_ROBOT_ID

    self.robot_pose = None
    self.puck_pose = None
    self.goal_pose = None

    self.lookahead_l = 0.10
    self.kp_position = 0.55
    self.max_linear_speed = 0.28
    self.max_angular_speed = 0.75
    self.position_tolerance = 0.07

    # Shooting distance
    self.STICK_RADIUS = 0.70

    # Horizontal offset
    self.CONTACT_RADIUS_CORRECTION = 0.02

    # The robot first navigates to a point exactly this far from the puck.
    self.SHOT_PRE_SWING_DISTANCE = SHOT_PRE_SWING_DISTANCE

    self.SHOT_PRE_ROTATION_ANGLE = SHOT_PRE_ROTATION_ANGLE
    self.SHOT_PRE_ROTATION_SPEED = SHOT_PRE_ROTATION_SPEED
    self.SHOT_SHOT_ROTATION_ANGLE = SHOT_SHOT_ROTATION_ANGLE
    self.SHOT_SHOT_ROTATION_SPEED = SHOT_SHOT_ROTATION_SPEED
    self.SHOT_PRE_ROTATION_SIGN = SHOT_PRE_ROTATION_SIGN
    self.SHOT_SHOT_ROTATION_SIGN = SHOT_SHOT_ROTATION_SIGN

    if self.SHOT_PRE_ROTATION_SIGN not in (-1.0, 1.0):
        raise ValueError("PRE_ROTATION_SIGN must be +1.0 or -1.0")
    if self.SHOT_SHOT_ROTATION_SIGN != -self.SHOT_PRE_ROTATION_SIGN:
        raise ValueError(
            "SHOT_ROTATION_SIGN must be opposite PRE_ROTATION_SIGN"
        )

    # Slow alignment
    self.heading_tolerance = math.radians(3.0)
    self.kp_heading = 1.5
    self.max_align_angular_speed = 0.55

    # Final low-speed approach
    self.max_final_approach_speed = 0.10
    self.min_final_approach_speed = 0.025
    self.final_speed_kp = 0.60
    self.final_heading_kp = 0.8
    self.max_final_heading_correction = 0.20
    self.final_position_tolerance = 0.04

    # Final-position
    self.SHOT_ROBOT_TO_PUCK_DISTANCE = SHOT_ROBOT_TO_PUCK_DISTANCE
    self.SHOT_SHOT_SIDE = SHOT_SHOT_SIDE
    self.SHOT_FINAL_STOP_EARLY_OFFSET = SHOT_FINAL_STOP_EARLY_OFFSET
    self.SHOT_SHOT_DIRECTION_OFFSET = SHOT_SHOT_DIRECTION_OFFSET
    self.stop_line_margin = 0.02
    self.max_safe_lateral_error = 0.4

    self.SHOT_LOWER_STICK_TIME = SHOT_LOWER_STICK_TIME
    self.SHOT_STOP_SETTLE_TIME = SHOT_STOP_SETTLE_TIME
    self.SHOT_STICK_GROUND_SETTLE_TIME = SHOT_STICK_GROUND_SETTLE_TIME
    self.SHOT_MAX_PRE_ROTATION_TIME = SHOT_MAX_PRE_ROTATION_TIME
    self.SHOT_MAX_SHOT_ROTATION_TIME = SHOT_MAX_SHOT_ROTATION_TIME

    self.stage = "wait_for_poses"
    self.last_status = None

    self.pre_swing_target = None
    self.swing_center_target = None
    self.contact_heading = None
    self.approach_heading = None
    self.pre_rotation_start_heading = None
    self.backward_heading = None

    self.stage_start_time = None
    self.rotation_start_time = None
    self.previous_rotation_theta = None
    self.accumulated_rotation_angle = 0.0

    best_effort_qos = QoSProfile(
        reliability=ReliabilityPolicy.BEST_EFFORT,
        durability=DurabilityPolicy.VOLATILE,
        depth=1,
    )

    self.cmd_pub = self.create_publisher(
        Twist,
        f"/robot{self.robot_id}/cmd_vel",
        10,
    )

    self.arm_pub = self.create_publisher(
        Vector3,
        f"/robot{self.robot_id}/cmd_arm",
        10,
    )

    self.create_subscription(
        PoseStamped,
        SHOT_ROBOT_POSE_TOPIC,
        self.robot_pose_callback,
        best_effort_qos,
    )

    self.create_subscription(
        PoseStamped,
        SHOT_PUCK_POSE_TOPIC,
        self.puck_pose_callback,
        best_effort_qos,
    )

    self.create_subscription(
        PoseStamped,
        SHOT_GOAL_POSE_TOPIC,
        self.goal_pose_callback,
```

```

        best_effort_qos,
    )

    self.timer = self.create_timer(0.05, self.control_loop)

    self.get_logger().info(
        f"Rotate-shot controller created for robot {self.robot_id}."
    )
    self.get_logger().info(f"Robot pose topic: {SHOT_ROBOT_POSE_TOPIC}")
    self.get_logger().info(f"Puck pose topic: {SHOT_PUCK_POSE_TOPIC}")
    self.get_logger().info(f"Goal pose topic: {SHOT_GOAL_POSE_TOPIC}")
    self.get_logger().info(
        f"Robot-to-puck distance: "
        f"{self.SHOT_ROBOT_TO_PUCK_DISTANCE:.3f} m"
    )
    self.get_logger().info(
        f"Shot side: {self.SHOT_SHOT_SIDE:+.0f}"
    )
    self.get_logger().info(
        f"Final stop early offset: "
        f"{self.SHOT_FINAL_STOP_EARLY_OFFSET:.3f} m"
    )
    self.get_logger().info(
        f"Shot-direction offset: "
        f"{self.SHOT_SHOT_DIRECTION_OFFSET:.3f} m"
    )
)

# -----
# Pose callbacks and utilities
# -----

def robot_pose_callback(self, msg):
    self.robot_pose = msg

def puck_pose_callback(self, msg):
    self.puck_pose = msg

def goal_pose_callback(self, msg):
    self.goal_pose = msg

@staticmethod
def yaw_from_quaternion(q):
    siny_cosp = 2.0 * (q.w * q.z + q.x * q.y)
    cosy_cosp = 1.0 - 2.0 * (q.y * q.y + q.z * q.z)
    return math.atan2(siny_cosp, cosy_cosp)

@staticmethod
def wrap_angle(angle):
    return math.atan2(math.sin(angle), math.cos(angle))

@staticmethod
def clamp(value, minimum, maximum):
    return max(min(value, maximum), minimum)

def robot_xytheta(self):
    x = self.robot_pose.pose.position.x
    y = self.robot_pose.pose.position.y
    theta = self.yaw_from_quaternion(
        self.robot_pose.pose.orientation
    )
    return x, y, theta

def log_once(self, message):
    if message != self.last_status:
        self.get_logger().info(message)
        self.last_status = message

def stop_robot(self):
    self.cmd_pub.publish(Twist())

def hard_stop_robot(self, repetitions=8, delay=0.02):
    for _ in range(repetitions):
        self.stop_robot()
        time.sleep(delay)

def stop_arm(self):
    self.arm_pub.publish(Vector3())

def move_arm_z(self, z_speed, duration):
    cmd = Vector3()
    cmd.z = float(z_speed)

    period = 1.0 / SHOT_ARM_PUBLISH_RATE
    end_time = time.monotonic() + duration

    try:
        while rclpy.ok() and time.monotonic() < end_time:
            self.arm_pub.publish(cmd)
            rclpy.spin_once(self, timeout_sec=0.0)
            time.sleep(period)
    finally:
        self.stop_arm()
        time.sleep(0.3)

# -----
# Shot geometry
# -----

def calculate_shot_geometry(self):
    puck_x = self.puck_pose.pose.position.x
    puck_y = self.puck_pose.pose.position.y

    goal_x = self.goal_pose.pose.position.x
    goal_y = self.goal_pose.pose.position.y

    shot_dx = goal_x - puck_x
    shot_dy = goal_y - puck_y
    shot_length = math.hypot(shot_dx, shot_dy)

    if shot_length < 1e-6:
        self.get_logger().error(
            "Puck and goal positions are too close to define a shot direction."
        )
        return False

    # Shooting direction
    ux = shot_dx / shot_length
    uy = shot_dy / shot_length

    # Normal to shooting direction
    normal_x = uy
    normal_y = -ux

```

```

        if self.SHOT_SHOT_SIDE not in (-1.0, 1.0):
            self.get_logger().error(
                "SHOT_SIDE must be either +1.0 or -1.0."
            )
            return False

    assumed_puck_x = (
        puck_x + self.SHOT_SHOT_DIRECTION_OFFSET * ux
    )
    assumed_puck_y = (
        puck_y + self.SHOT_SHOT_DIRECTION_OFFSET * uy
    )

    self.swing_center_target = (
        assumed_puck_x
        + self.SHOT_SHOT_SIDE
        * self.SHOT_ROBOT_TO_PUCK_DISTANCE
        * normal_x,
        assumed_puck_y
        + self.SHOT_SHOT_SIDE
        * self.SHOT_ROBOT_TO_PUCK_DISTANCE
        * normal_y,
    )

    self.pre_swing_target = (
        assumed_puck_x
        + self.SHOT_SHOT_SIDE
        * self.SHOT_PRE_SWING_DISTANCE
        * normal_x,
        assumed_puck_y
        + self.SHOT_SHOT_SIDE
        * self.SHOT_PRE_SWING_DISTANCE
        * normal_y,
    )

    # Final alignment points
    radial_x = assumed_puck_x - self.swing_center_target[0]
    radial_y = assumed_puck_y - self.swing_center_target[1]

    self.contact_heading = math.atan2(radial_y, radial_x)
    self.pre_rotation_start_heading = self.contact_heading

    self.backward_heading = self.wrap_angle(
        self.pre_rotation_start_heading + math.pi
    )

    approach_dx = (
        self.swing_center_target[0] - self.pre_swing_target[0]
    )
    approach_dy = (
        self.swing_center_target[1] - self.pre_swing_target[1]
    )
    self.approach_heading = math.atan2(
        approach_dy,
        approach_dx,
    )

    self.get_logger().info(
        "Shot geometry calculated: "
        f"real_puck=({puck_x:.3f}, {puck_y:.3f}), "
        f"assumed_puck=({assumed_puck_x:.3f}, "
        f"{assumed_puck_y:.3f}), "
        f"goal=({goal_x:.3f}, {goal_y:.3f})"
    )
    self.get_logger().info(
        f"pre_swing=({self.pre_swing_target[0]:.3f}, "
        f"{self.pre_swing_target[1]:.3f}), "
        f"swing_center=({self.swing_center_target[0]:.3f}, "
        f"{self.swing_center_target[1]:.3f})"
    )
    self.get_logger().info(
        f"contact_heading="
        f"{math.degrees(self.contact_heading):.1f} deg, "
        f"pre_rotation=180.0 deg, shot_rotation=270.0 deg"
    )

    return True

# -----
# Navigation
# -----

def drive_to_point(self, target_x, target_y, tolerance):
    x, y, theta = self.robot_xytheta()

    px = x + self.lookahead_l * math.cos(theta)
    py = y + self.lookahead_l * math.sin(theta)

    ex = target_x - px
    ey = target_y - py
    distance = math.hypot(ex, ey)

    if distance < tolerance:
        self.stop_robot()
        return True

    pdot_x = self.kp_position * ex
    pdot_y = self.kp_position * ey

    v = (
        math.cos(theta) * pdot_x
        + math.sin(theta) * pdot_y
    )

    w = (
        -math.sin(theta) * pdot_x
        + math.cos(theta) * pdot_y
    ) / self.lookahead_l

    v = self.clamp(
        v,
        -self.max_linear_speed,
        self.max_linear_speed,
    )

    w = self.clamp(
        w,
        -self.max_angular_speed,
        self.max_angular_speed,
    )

```

```

cmd = Twist()
cmd.linear.x = v
cmd.angular.z = w
self.cmd_pub.publish(cmd)

return False

def align_to_heading(self, desired_heading):
    _, _, theta = self.robot_xytheta()

    heading_error = self.wrap_angle(
        desired_heading - theta
    )

    if abs(heading_error) < self.heading_tolerance:
        self.stop_robot()
        return True

    w = self.kp_heading * heading_error
    w = self.clamp(
        w,
        -self.max_align_angular_speed,
        self.max_align_angular_speed,
    )

    cmd = Twist()
    cmd.angular.z = w
    self.cmd_pub.publish(cmd)

    return False

def drive_backward_to_swing_center(self):
    """
    Reverse toward the final shooting position with closed-loop position
    correction.
    """
    x, y, theta = self.robot_xytheta()
    target_x, target_y = self.swing_center_target

    forward_x = math.cos(self.approach_heading)
    forward_y = math.sin(self.approach_heading)

    stop_x = (
        target_x
        - self.SHOT_FINAL_STOP_EARLY_OFFSET * forward_x
    )
    stop_y = (
        target_y
        - self.SHOT_FINAL_STOP_EARLY_OFFSET * forward_y
    )

    center_error_x = stop_x - x
    center_error_y = stop_y - y

    remaining_forward = (
        center_error_x * forward_x
        + center_error_y * forward_y
    )

    lateral_error = (
        -center_error_x * forward_y
        + center_error_y * forward_x
    )

    center_distance = math.hypot(
        center_error_x,
        center_error_y,
    )

    if remaining_forward <= self.stop_line_margin:
        self.hard_stop_robot()
        self.get_logger().info(
            "Backward swing-center stopping line reached: "
            f"remaining_forward={remaining_forward:.3f} m, "
            f"lateral_error={lateral_error:.3f} m"
        )
        return True

    if center_distance <= self.final_position_tolerance:
        self.hard_stop_robot()
        return True

    if abs(lateral_error) > self.max_safe_lateral_error:
        self.hard_stop_robot()
        self.get_logger().error(
            "Backward approach stopped because lateral error is unsafe: "
            f"{lateral_error:.3f} m"
        )
        self.stage = "error"
        return False

    # Rear virtual point
    rear_x = x - self.lookahead_l * math.cos(theta)
    rear_y = y - self.lookahead_l * math.sin(theta)

    # Matching rear virtual point at the desired final pose.
    rear_target_x = (
        stop_x
        - self.lookahead_l * math.cos(self.backward_heading)
    )
    rear_target_y = (
        stop_y
        - self.lookahead_l * math.sin(self.backward_heading)
    )

    ex = rear_target_x - rear_x
    ey = rear_target_y - rear_y

    pdot_x = self.kp_position * ex
    pdot_y = self.kp_position * ey

    v = (
        math.cos(theta) * pdot_x
        + math.sin(theta) * pdot_y
    )

    w = (
        math.sin(theta) * pdot_x
        - math.cos(theta) * pdot_y
    ) / self.lookahead_l

    v = self.clamp(
        v,

```

```

        -self.max_final_approach_speed,
        -self.min_final_approach_speed,
    )

    w = self.clamp(
        w,
        -self.max_final_heading_correction,
        self.max_final_heading_correction,
    )

    cmd = Twist()
    cmd.linear.x = v
    cmd.angular.z = w
    self.cmd_pub.publish(cmd)

    return False

# -----
# Rotations
# -----

def start_measured_rotation(self):
    _, _, theta = self.robot_xytheta()
    self.previous_rotation_theta = theta
    self.accumulated_rotation_angle = 0.0
    self.rotation_start_time = self.get_clock().now()

def update_measured_rotation(
    self,
    direction_sign,
    target_angle,
    angular_speed,
    timeout,
    label,
):
    _, _, theta = self.robot_xytheta()

    delta_theta = self.wrap_angle(
        theta - self.previous_rotation_theta
    )
    self.previous_rotation_theta = theta

    signed_increment = direction_sign * delta_theta
    if signed_increment > 0.0:
        self.accumulated_rotation_angle += signed_increment

    elapsed = (
        self.get_clock().now() - self.rotation_start_time
    ).nanoseconds / 1e9

    if self.accumulated_rotation_angle >= target_angle:
        self.hard_stop_robot()
        self.get_logger().info(
            f"{label} reached "
            f"{math.degrees(self.accumulated_rotation_angle):.1f} deg."
        )
        return True

    if elapsed >= timeout:
        self.hard_stop_robot()
        self.get_logger().error(
            f"{label} timed out at "
            f"{math.degrees(self.accumulated_rotation_angle):.1f} deg."
        )
        self.stage = "error"
        return False

    cmd = Twist()
    cmd.linear.x = 0.0
    cmd.angular.z = direction_sign * angular_speed
    self.cmd_pub.publish(cmd)
    return False

# -----
# State machine
# -----

def control_loop(self):
    if self.stage == "wait_for_poses":
        self.stop_robot()

        if self.robot_pose is None:
            self.log_once("Waiting for robot pose...")
            return

        if self.puck_pose is None:
            self.log_once("Waiting for puck pose...")
            return

        if self.goal_pose is None:
            self.log_once("Waiting for goal pose...")
            return

        self.last_status = None
        self.stage = "calculate_shot"

    if self.stage == "calculate_shot":
        self.stop_robot()

        if not self.calculate_shot_geometry():
            self.stage = "error"
            return

        self.stage = "go_to_pre_swing"
        self.last_status = None
        return

    if self.stage == "go_to_pre_swing":
        self.log_once("Stage: GO_TO_PRE_SWING")

        reached = self.drive_to_point(
            self.pre_swing_target[0],
            self.pre_swing_target[1],
            self.position_tolerance,
        )

        if reached:
            self.get_logger().info(
                "Reached the pre-swing point."
            )
            self.stage = "align_for_pre_rotation"
            self.last_status = None

```

```

        return

    if self.stage == "align_for_pre_rotation":
        self.log_once("Stage: ALIGN_FOR_PRE_ROTATION")

        aligned = self.align_to_heading(
            self.pre_rotation_start_heading
        )

        if aligned:
            self.hard_stop_robot()
            self.get_logger().info(
                "At the 1 m pre-swing point; raised stick aligned toward puck."
            )
            self.start_measured_rotation()
            self.stage = "pre_rotate_180"
            self.last_status = None
            return

    if self.stage == "pre_rotate_180":
        self.log_once("Stage: PRE_ROTATE_180")

        completed = self.update_measured_rotation(
            direction_sign=self.SHOT_PRE_ROTATION_SIGN,
            target_angle=self.SHOT_PRE_ROTATION_ANGLE,
            angular_speed=self.SHOT_PRE_ROTATION_SPEED,
            timeout=self.SHOT_MAX_PRE_ROTATION_TIME,
            label="Raised-stick pre-rotation",
        )

        if self.stage == "error":
            return

        if completed:
            self.stage_start_time = self.get_clock().now()
            self.stage = "settle_after_pre_rotation"
            self.last_status = None
            return

    if self.stage == "settle_after_pre_rotation":
        self.log_once("Stage: SETTLE_AFTER_PRE_ROTATION")
        self.stop_robot()

        elapsed = (
            self.get_clock().now() - self.stage_start_time
        ).nanoseconds / 1e9

        if elapsed >= self.SHOT_STOP_SETTLE_TIME:
            self.stage = "backward_to_swing_center"
            self.last_status = None
            return

    if self.stage == "backward_to_swing_center":
        self.log_once("Stage: BACKWARD_TO_SWING_CENTER")

        reached = self.drive_backward_to_swing_center()

        if self.stage == "error":
            return

        if reached:
            self.get_logger().info(
                "Reached the final swing-center position while reversing."
            )
            self.stage = "lower_stick"
            self.last_status = None
            return

    if self.stage == "lower_stick":
        self.log_once("Stage: LOWER_STICK")
        self.hard_stop_robot()

        self.get_logger().info(
            "Lowering arm so the stick touches the floor."
        )
        self.move_arm_z(
            SHOT_LOWER_ARM_Z_SPEED,
            self.SHOT_LOWER_STICK_TIME,
        )

        self.stage_start_time = self.get_clock().now()
        self.stage = "settle_stick_on_ground"
        self.last_status = None
        return

    if self.stage == "settle_stick_on_ground":
        self.log_once("Stage: SETTLE_STICK_ON_GROUND")
        self.stop_robot()
        self.stop_arm()

        elapsed = (
            self.get_clock().now() - self.stage_start_time
        ).nanoseconds / 1e9

        if elapsed >= self.SHOT_STICK_GROUND_SETTLE_TIME:
            self.start_measured_rotation()
            self.get_logger().info(
                "Starting fast 270-degree shot rotation."
            )
            self.stage = "fast_shot_rotate_180"
            self.last_status = None
            return

    if self.stage == "fast_shot_rotate_180":
        self.log_once("Stage: FAST_SHOT_ROTATE_180")

        completed = self.update_measured_rotation(
            direction_sign=self.SHOT_SHOT_ROTATION_SIGN,
            target_angle=self.SHOT_SHOT_ROTATION_ANGLE,
            angular_speed=self.SHOT_SHOT_ROTATION_SPEED,
            timeout=self.SHOT_MAX_SHOT_ROTATION_TIME,
            label="Shot rotation",
        )

        if self.stage == "error":
            return

        if completed:
            self.stage = "done"
            self.last_status = None
            return

    if self.stage == "done":

```

```

        self.log_once("Stage: DONE")
        self.stop_robot()
        self.stop_arm()
        return

    if self.stage == "error":
        self.log_once("Stage: ERROR")
        self.stop_robot()
        self.stop_arm()

# =====
# COMPLETE-CONTROLLER
# =====
def run_grab_stage():
    node = NavigateAndGrabStick()

    try:
        prepared = node.prepare_arm_for_navigation()
        if not prepared:
            node.get_logger().error(
                "Full sequence cancelled because preparation failed."
            )
            node.stage = "failed"
            return False

        node.stage = "go_to_pre_approach"
        node.get_logger().info(
            f"Starting navigation to stick {node.stick_id}."
        )

        while (
            rclpy.ok()
            and node.stage not in ("navigation_done", "failed")
        ):
            rclpy.spin_once(node, timeout_sec=0.1)

        if node.stage != "navigation_done":
            return False

        node.hard_stop_robot()
        time.sleep(0.5)

        grabbed = node.grab_stick()
        if not grabbed:
            node.stage = "failed"
            node.get_logger().error(
                "Robot reached the stick, but the grab sequence failed."
            )
            return False

        node.stage = "return_to_pre_approach"
        node.last_status = None
        node.get_logger().info(
            "Stick grabbed successfully. Reversing to the original "
            "pre-approach point before starting the shooting stage."
        )

        while (
            rclpy.ok()
            and node.stage not in ("returned_to_pre_approach", "failed")
        ):
            rclpy.spin_once(node, timeout_sec=0.1)

        if node.stage != "returned_to_pre_approach":
            node.get_logger().error(
                "Could not return safely to the pre-approach point."
            )
            return False

        node.hard_stop_robot()
        node.get_logger().info(
            "Safe clearance from the stick base achieved. "
            "Starting shooting stage next."
        )
        return True

    finally:
        node.hard_stop_robot()
        node.stop_arm()
        node.destroy_node()

def run_shot_stage():
    node = RotateShotController()

    try:
        node.get_logger().info(
            "Shooting stage started; waiting for robot, puck, and goal poses."
        )

        while rclpy.ok() and node.stage not in ("done", "error"):
            rclpy.spin_once(node, timeout_sec=0.1)

        if node.stage == "done":
            node.get_logger().info(
                "Complete grab-and-shoot sequence finished successfully."
            )
            return True

        node.get_logger().error("Shooting stage ended in ERROR.")
        return False

    finally:
        node.hard_stop_robot()
        node.stop_arm()
        node.destroy_node()

def main(args=None):
    rclpy.init(args=args)

    try:
        if not run_grab_stage():
            return

        # Preserve the physical arm/gripper state while changing ROS nodes.
        time.sleep(0.75)
        run_shot_stage()

    except KeyboardInterrupt:
        print("\nComplete hockey sequence interrupted by user.")

```

```
finally:
    if rclpy.ok():
        rclpy.shutdown()

if __name__ == "__main__":
    main()
```